

I'll conduct a comprehensive scan of your codebase to understand its architecture, components, and functionality. Let me start by exploring the key areas.

[16 tools called]

Let me continue exploring the key source files to understand the full architecture:

[42 tools called]

Now I have a comprehensive understanding of your codebase. Here's my detailed summary:

Voyager - Comprehensive Codebase Summary

1. Overview

Voyager is a sophisticated **investment thesis management and portfolio analytics platform** that combines:

- **LLM-powered intelligence** (via OpenAI) for thesis review, Q&A, and daily summaries
- **Quantitative backtesting** (via Backtrader) for strategy evaluation
- **Semantic memory** (via pgvector embeddings) for observation recall
- **Risk management** (factor models, scenarios, VaR)
- **Paper trading execution** for thesis implementation

The system is built around **investment theses** as first-class entities, with observations, trades, and evaluations linked back to specific theses.

2. Architecture Layers

2.1 Data Layer

Component	Technology	Purpose
-----	-----	-----
Database	PostgreSQL + pgvector	Stores theses, observations, trades, evaluations, alerts
Market Data	TwelveData API	Stock/ETF/Crypto price data
Economic Data	FRED API	Macro indicators (yields, inflation, etc.)
Embeddings	OpenAI `text-embedding-3-large`	1536-dim vectors for semantic search

Schema Structure (3 SQL files):

- `voyager\_schema.sql` - Core: `market\_data`, `econ\_data`
- `phase4\_schema.sql` - Domain: `thesis`, `observation` (with pgvector), `trade`, `scenario`
- `e4\_schema.sql` - Evaluation: `thesis\_evaluation`, `alert`, `daily\_summary`

2.2 Domain Models (`src/voyager/models/`)

Model	Purpose
-----	-----
`Thesis`	Investment hypothesis with drivers, disconfirmers, expression legs

| `Observation` | Time-stamped notes with thesis refs, sentiment, categories, actionability
|
| `Trade` | Executed positions (simulated/real) linked to theses |

| `ThesisEvaluationResult` | Quant metrics: performance, timeseries, risk metrics, scenarios |

| `ThesisReview` | LLM critique, questions, risk flags |

| `PortfolioSnapshot` | Current positions and totals |

| `TradePlan` / `TradeLeg` | Execution plan for thesis implementation |

****Key Enums:****

- `ThesisStatus` : ACTIVE, CLOSED, WATCHLIST
- `Direction` : LONG, SHORT
- `Sentiment` : VERY_BULLISH → VERY_BEARISH, ANXIOUS, CONFIDENT
- `TradeType` : SIMULATED, REAL

2.3 Repository Layer (`src/voyager/repositories/`)

Seven repositories providing CRUD operations:

- `ThesisRepository` - thesis CRUD + list_active()
- `ObservationRepository` - with embedding vector support
- `TradeRepository` - list_by_thesis(), list_all()
- `EvaluationRepository` - thesis evaluation persistence
- `AlertRepository` - alert management by date/thesis
- `DailySummaryRepository` - daily portfolio summaries
- `ScenarioRepository` - scenario definitions

3. Core Subsystems

3.1 Session Orchestrator (`src/voyager/session/`)

****Central pipeline for all LLM interactions:****

`,`,

User Text → Ingestion → Memory Retrieval → Risk Snapshot → Prompt Building → LLM
Call → Logging → Response

`,`,

****Session Modes:****

- `STANDARD` - General Q&A
- `ANALYST` - Thesis review mode
- `CONCISE` - Brief responses (daily summary)

****Session Options control:****

- Memory inclusion (`use\_memory`, `skip\_memory`)
- Risk context (`use\_risk`, `skip\_risk`)
- Observation ingestion (`skip\_ingest`)

3.2 Intelligence Layer (`src/voyager/intelligence/`)

****Context Building (`context/`):****

- `DataAccess` - Unified data access facade for all repositories

- `ContextBuilder` - Assembles thesis context from multiple sources
- `portfolio\_adapter` - Builds portfolio snapshots from trades
- `concentration` - Computes position concentration metrics
- `factors` - Factor exposure calculations
- `macro\_adapter` - Macro regime classification

****Higher-level Modules:****

- `OrchestratorClient` - Phase-6 wrapper around SessionOrchestrator
- `ThesisReviewer` - Builds context and runs ANALYST mode review
- `IntuitionQAEngine` - RAG-style Q&A using recalled observations
- `thesis\_consistency` - Cross-thesis analysis
- `commentary` - Narrative generation

3.3 LLM Tools (`src/voyager/llm/`)

****Four core tool functions:****

1. `llm\_review\_thesis()` - Critique thesis against evaluation
2. `llm\_daily\_summary()` - Portfolio manager briefing
3. `llm\_cross\_theses()` - Multi-thesis overlap/contradiction analysis
4. `llm\_query\_intuition()` - Q&A against observation context

****Output DTOs:****

- `ThesisReview` : critique, questions, risk_flags, insufficient_context
- `DailySummary` : key_narratives, risk_highlights, thesis_references
- `CrossThesisReport` : overlaps, contradictions, gaps
- `IntuitionAnswer` : answer, references (validated), insufficient_context

****Features:****

- JSON extraction with salvaging (`extract\_json()`)
- Reference validation (filters hallucinated IDs)
- Metrics/latency tracking

3.4 Memory System (`src/voyager/memory/`)

****Semantic recall using pgvector:****

- `embed\_observation\_text()` - OpenAI embedding generation
- `search\_similar\_observations()` - K-NN search with filters (time, categories, sentiment)
- `MemoryService` - High-level recall API:
 - `recall\_similar\_text()` - Free-text query
 - `recall\_for\_observation()` - Find related observations
 - `recall\_for\_thesis()` - Find observations relevant to thesis

3.5 Quant Engine (`src/voyager/quant\_engine/`)

****Backtrader-based backtesting framework:****

****Core:****

- `StrategyBase` - Abstract base with `compute\_target\_weights() → Dict[symbol, weight]`
- `build\_cerebro()` / `run\_cerebro()` - Engine configuration with analyzers
- `VoyagerPandasData` - Custom data feed adapter

****Strategies:****

- `BuyAndHoldFirstSymbol` - Simple baseline
- `GoldRealYieldsStrategy` - Long gold when real yields < MA
- `USDDivergenceStrategy` - USD strength signals
- `CurveSteepenerStrategy` - Yield curve positioning

****Data Loading:****

- `load\_price\_data()` - From `market\_data` table
- `load\_econ\_series()` - From `econ\_data` table

****Interface:****

- `run\_backtest(strategy\_id, params)` → `BacktestResult` with returns, orders, trades, metrics

3.6 Risk System (`src/voyager/risk/`)

****Comprehensive risk analytics:****

- `compute\_risk\_metrics()` - VaR, volatility, Sharpe, drawdown
- `run\_factor\_regression()` - Factor model fitting
- `run\_scenarios()` - Shock scenario analysis
- `compute\_risk\_rails()` - Concentration flags, correlation clusters, regime warnings
- `build\_risk\_report()` - Master entrypoint combining all components

****Schemas:****

- `RiskMetrics` - VaR, volatility, Sharpe, max_drawdown
- `FactorModel` - Factor exposures, R^2
- `ScenarioResult` - Scenario name, PnL impact

- `RiskRails` - Rails/flags for concentration, correlations, regimes
- `RiskReport` - Full composite report

3.7 Evaluation Service (`src/voyager/evaluation/`)

****Thesis evaluation pipeline:****

1. Validate thesis expression legs
2. Determine backtest window
3. Fetch aligned price history via `PriceSource`
4. Compute daily returns and equity curve
5. Calculate performance metrics (total return, CAGR, volatility, Sharpe, max drawdown)
6. Calculate risk metrics (max weight, VaR 95%, max drawdown)
7. Run scenarios (All -10%, All +10%)
8. Return `ThesisEvaluationResult`

3.8 Execution System (`src/voyager/execution/`)

****Paper trading implementation:****

- `PaperExecutionAdapter` - Converts theses to trade plans and executes
- `TradePlan` / `TradeLeg` - Execution DTOs
- `ThesisPnL` - P&L calculation for thesis positions

****Current limitations (E5 v1):****

- Long-only trades
- No slippage/fees
- No leverage/margin checks

4. API Layer (`src/voyager/api/`)

****FastAPI application with multiple routers:****

Router	Prefix	Purpose
-----	-----	-----
`session_router`	`/session`	Core session management
`intelligence_router`	`/intelligence`	Thesis review, Q&A
`e4_session_router`	`/`	Evaluation and daily update sessions
`ui_router`	`/ui`	UI-facing endpoints
`diagnostics_router`	`/`	LLM diagnostics
`memory_app`	`/memory`	Memory API

****Dependency Injection (`deps.py`):****

- `get\_data\_access\_instance()` - Singleton DataAccess
- `get\_price\_source\_instance()` - PriceSource (currently mock)
- `get\_orchestrator\_client\_instance()` - OrchestratorClient with OpenAI

5. Session Workflows (`src/voyager/sessions/`)

`ThesisEvaluationSession`

****Atomic thesis evaluation:****

1. Load thesis by ID
2. Run E2 quant evaluation
3. Run E3 LLM review
4. (Optional) Generate trade plan
5. Persist atomically (only if both succeed)
6. Return `ThesisEvaluationSessionResult`

`DailyUpdateSession`

****Portfolio daily update with graceful degradation:****

1. Load active theses and portfolio
2. Detect alerts from observations
3. Persist alerts (before LLM call)
4. Build DailyContext
5. Run LLM summary (graceful failure → insufficient_context=True)
6. Persist and return result

`CrossThesisSession`

- Analyzes relationships between multiple theses

6. UI Layer (`ui/app.py`)

****Streamlit dashboard with 5 tabs:****

1. **Theses** - List with status, P&L, risk flags, evaluate/view actions
2. **Thesis Detail** - Full view with expression legs, evaluation, LLM review, trade plan, approve action
3. **Portfolio** - Positions, totals, P&L, concentration metrics, thesis allocation
4. **Alerts & Daily Runs** - Recent alerts, run daily update, LLM diagnostics
5. **Observations & Q&A** - Recent observations, intuition query interface

7. Ingestion Pipeline (`src/voyager/ingest/`)

Validation + persistence for all domain objects:

- `ingest\_thesis()` - Validate and insert thesis
- `ingest\_observation()` - With optional embedding
- `ingest\_observation\_with\_embedding()` - Full pipeline with OpenAI embedding
- `ingest\_trade()` - Validate and insert trade
- `ingest\_scenario()` - Validate and insert scenario

Uses `llm\_validation/` for schema validation via Pydantic.

8. Development Phases (from docs/v2/)

The project evolved through **6 epochs (E1-E6)**:

Epoch	Focus
-----	-----
E1	Data access layer, portfolio snapshots
E2	Thesis evaluation service (quant metrics)
E3	LLM tools (review, summary, cross-thesis, Q&A)
E4	Session workflows, repositories, persistence
E5	Execution adapter, trade plans, P&L
E6	UI harness, full integration

9. Key Configuration

****Environment Variables:****

- `VOYAGER\_DB\_URL` - PostgreSQL connection string
- `TWELVEDATA\_API\_KEY` - Market data
- `FRED\_API\_KEY` - Economic data
- `OPENAI\_API\_KEY` - LLM and embeddings
- `OPENAI\_MODEL` - Default: gpt-4
- `OPENAI\_EMBEDDING\_MODEL` - Default: text-embedding-3-large

10. Test Structure (`tests/`)

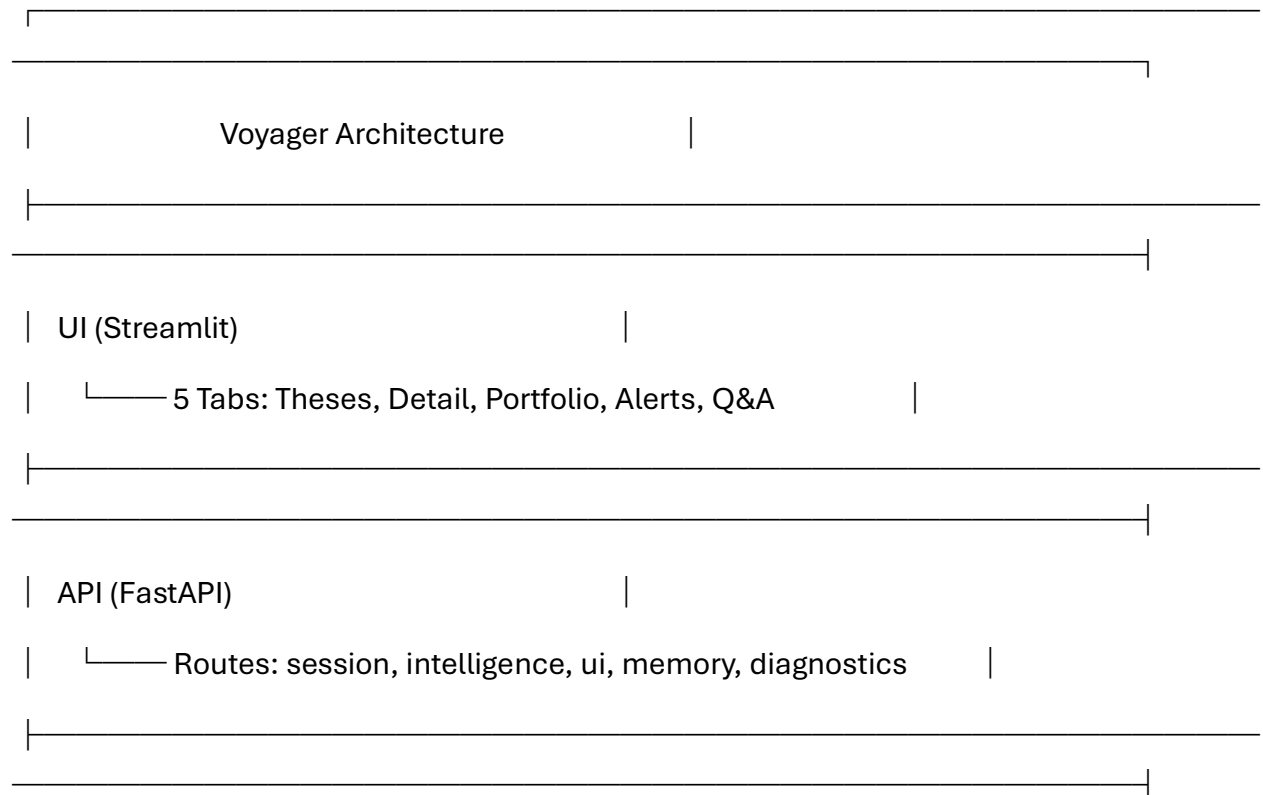
Organized by phase/epoch:

- `tests/e3/` - LLM tool tests
- `tests/e4/` - Session and repository tests
- `tests/e5/` - Execution and P&L tests
- `tests/phase6/` - Intelligence module tests
- `tests/phase7/`, `phase8/`, `phase9/` - Additional integration tests

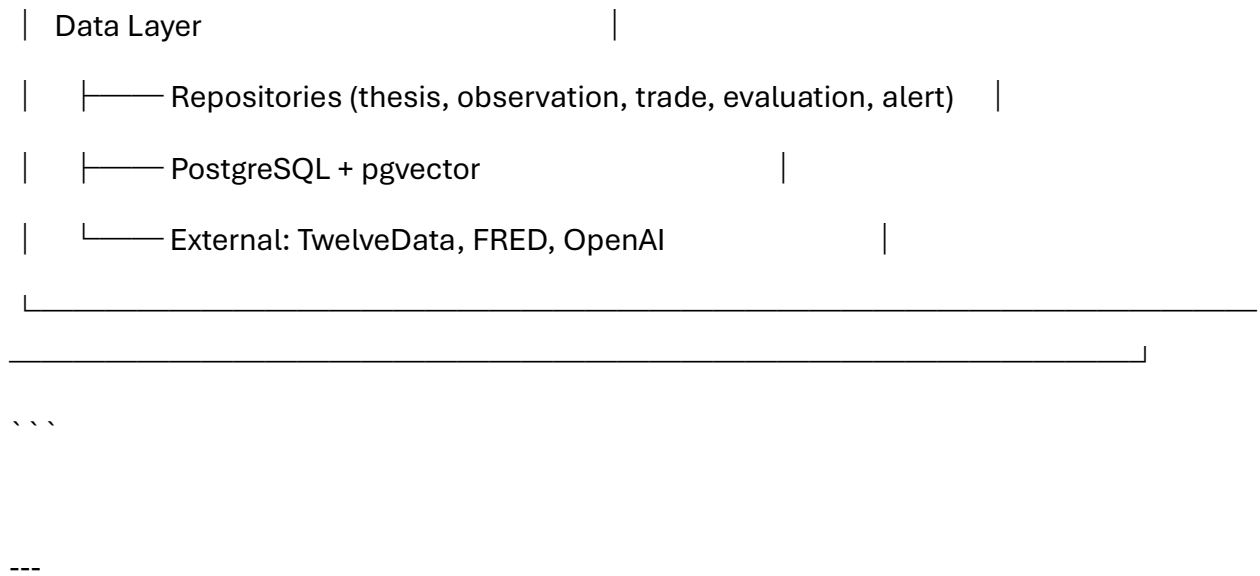
Fixtures in `conftest.py` provide database engine and table cleanup.

Summary Diagram

...







Would you like me to dive deeper into any specific component, or do you have questions about how particular subsystems interact?